Programming Assignment 1

Nuruzzaman Rahat

October 11, 2026

1 Executive Summary

The aim of this programming exercise is to develop and test different matrix storage and computation techniques that can reduce both memory usage and computational cost.Four subroutines were implemented: (1) computing (w = Lv) when (L) is a unit lower-triangular matrix stored in a dense 2-D array, (2) performing the same operation when (L) is stored in Compressed Sparse Row (CSR) format, (3) computing (w = Lv) for a unit lower-triangular banded matrix with two subdiagonals, and (4) computing the matrix product (M = LU), where the unit lower-triangular matrix (L) and upper-triangular matrix (U) are stored together in a single packed array. Each subroutine was tested using small hand-worked examples as well as randomized tests with matrix sizes up to (n=500). The results agreed with the reference calculations to within machine precision, confirming that the implementations were accurate and numerically reliable.

2 Statement of the Problem

The main objective of this assignment is to implement and test matrix operations by taking advantage of the special structure of triangular and banded matrices. The assignment focuses on reducing unnecessary storage and computational work by storing only the matrix elements that can contain nonzero values. Four subroutines are required: computing $w = Lv$ for a unit lower-triangular matrix stored as a dense 2-D array, computing the same product using compressed row or column storage, computing $w = Lv$ for a banded unit lower-triangular matrix with two subdiagonals, and computing the matrix product $M = LU$ when L and U are stored together in a single 2D array. The implemented routines must then be tested over different problem sizes and validated by comparing their results with known or reference solutions.

3 Description of the Mathematics

The mathematical foundation of this assignment is based on triangular matrices and structured matrix operations. A lower-triangular matrix L has all of its entries above the main diagonal equal to zero, while a unit lower-triangular matrix additionally has ones on its main diagonal.

Similarly, an upper-triangular matrix U has zeros below its main diagonal. These structural properties allow many unnecessary multiplication and addition operations to be avoided. For a matrix $L \in \mathbb{R}^{n\times n}$ and vector $v \in \mathbb{R}^n$, the matrix-vector product $w = Lv$ is defined componentwise as

$$w_i = \sum_{j=1}^{n} l_{ij} v_j.$$

Since L is lower triangular, only the terms with $j \leq i$ can be nonzero. For a unit lower-triangular matrix, $l_{ii} = 1$, so the product can be written as

$$w_i = v_i + \sum_{j=1}^{i-1} l_{ij} v_j.$$

This structure reduces the number of operations compared with a general dense matrix-vector multiplication. It requires only about $\frac{n^2}{2}$ multiplications and additions.

The same idea is applied when the lower-triangular matrix is stored in a compressed format. Instead of allocating storage for the entire matrix, the implementation keeps the entries that may be nonzero together with the information needed to locate them. The computation can then be carried out using only these stored values, making the method more efficient in terms of both storage and arithmetic operations.

A further reduction is possible for the banded matrix considered in the assignment. Here, the nonzero entries of L are limited to the two diagonals immediately below the main diagonal. As a result, the product $w = Lv$ has the form

$$w_1 = v_1,$$

$$w_2 = v_2 + l_{21} v_1,$$

and for $i \geq 3$,

$$w_i = v_i + l_{i,i-1} v_{i-1} + l_{i,i-2} v_{i-2}.$$

Thus, the amount of work per row remains bounded as n increases, giving an $O(n)$ computational procedure. The storage requirement is also linear in n, since only the two subdiagonals need to be retained.

The final mathematical operation is the formation of $M = LU$, where L is unit lower triangular and U is upper triangular. For an individual entry of the resulting matrix,

$$m_{ij} = \sum_{k=1}^{\min(i,j)} l_{ik} u_{kj}.$$

The upper-triangular and lower-triangular structures determine which terms in this summation can actually contribute. Consequently, the implementation avoids calculations involving entries that are known to be zero and uses the unit diagonal of L directly.

To assess the numerical correctness of these implementations, the computed vectors and matrices are compared against corresponding reference results. For a matrix-vector calculation, the difference is measured using

$$\epsilon_{\text{abs}} = \|w_{\text{comp}} - w_{\text{true}}\| \qquad \text{and} \qquad \epsilon_{\text{rel}} = \frac{\|w_{\text{comp}} - w_{\text{true}}\|}{\|w_{\text{true}}\|}.$$

For the matrix-matrix operation, the relative error is similarly calculated as

$$\epsilon_{\text{rel}} = \frac{\|M_{\text{comp}} - M_{\text{true}}\|}{\|M_{\text{true}}\|}.$$

Different matrix norms, including the 1-norm, infinity norm, and Frobenius norm, can be used for this comparison. Repeating these calculations over multiple matrix sizes and randomly generated test cases provides a quantitative way to evaluate the accuracy and consistency of the implemented routines.

4 Description of the Algorithm and Implementation

All four routines are written in basic Python, using explicit loops and singly or doubly indexed lists. No library routines are called inside any subroutine. NumPy and Matplotlib are used only in the testing code, to generate data, compute reference answers, and produce plots. The code uses 0-based indexing, while the mathematical descriptions below use 1-based indexing. Each routine takes the problem size n as an argument.

Subroutine 1: Unit lower triangular matrix–vector product (2-D array)

This routine computes $w \leftarrow Lv$ using the row-oriented inner product form. For each row i, the sum starts at v_i, which accounts for the unit diagonal without a multiplication. It then adds $l_{ij}v_j$ for $j < i$ only:

$$w_i = v_i + \sum_{j=1}^{i-1} l_{ij}v_j, \qquad i = 1, \ldots, n.$$

Entries on or above the diagonal of the array are never read. The cost is $n(n-1)/2$ multiplications and additions.

Subroutine 2: Unit lower triangular matrix–vector product (compressed rows)

The strictly lower part of L is stored row by row in a 1-D array of length $n(n-1)/2$:

$$\text{ARRAY} = [l_{21},\ l_{31},\ l_{32},\ l_{41},\ l_{42},\ l_{43},\ \ldots, l_{nn}].$$

The same inner product form as Subroutine 1 is used, since it matches this row-wise layout. A single counter moves forward through the array, so no index formula is needed. The operation count is the same as Subroutine 1, but the storage is reduced from n^2 to $n(n-1)/2$ locations.

Subroutine 3: Banded unit lower triangular matrix–vector product

Here L is unit lower triangular with nonzeros only on its two subdiagonals. Only these two diagonals are stored, in an 2-D array, so the storage is $2(n-1)$ locations. The unit diagonal and the zeros are not stored. Instead of looping over rows, the product is organized as three long vector operations:

$$\begin{aligned} w(1{:}n) &\leftarrow v(1{:}n), \\ w(2{:}n) &\leftarrow w(2{:}n) + l_{i,i-1} * v(1{:}n-1), \\ w(3{:}n) &\leftarrow w(3{:}n) + l_{i,i-2} * v(1{:}n-2), \end{aligned}$$

This follows the same idea as the tridiagonal example in Study Question 1.6 and uses $2n-3$ multiplications and additions, which is linear in n.

Subroutine 4: Matrix–matrix product $M = LU$ from a packed array

Both factors share a single $n \times n$ Matrix: the strictly lower part holds L, and the diagonal and upper part hold U. The unit diagonal of L and the zeros of both factors are never stored. The routine uses the inner product form. Since $l_{ik} = 0$ for $k > i$, $u_{kj} = 0$ for $k > j$, and $l_{ii} = 1$, each entry reduces to

$$m_{ij} = \begin{cases} \displaystyle\sum_{k=1}^{i-1} l_{ik}u_{kj} + u_{ij}, & i \le j, \\ \displaystyle\sum_{k=1}^{j-1} l_{ik}u_{kj} + l_{ij}u_{jj}, & i > j. \end{cases}$$

In the code, the inner loop runs over $k < \min(i, j)$ using only stored entries. The last term is then added separately. For $i \le j$ it is u_{ij}, added directly, so the unit diagonal never causes a multiplication. For $i > j$ it is the product $l_{ij}u_{jj}$. The cost is about $n^3/3$ multiplications and additions, compared with n^3 for a dense product.

Testing code

Each subroutine has the same structure:

1. a small hand-checked case with a known result;
2. a random case compared against NumPy (`L @ v`, or `L @ U` after unpacking L and U);
3. a sweep over many sizes n with 20 random trials each, summarized by the mean, maximum, and variance of the relative error, together with plots of error versus n and histograms.

5 Description of the Experimental Design and Results

To evaluate the correctness, numerical stability, and computational efficiency of each subroutine, a series of experiments was conducted in Python. The computed results were validated against dense reference solutions, and the relative errors were calculated to assess accuracy. For each experiment, the mean and maximum relative errors were recorded, along with histograms and error distributions, to analyze the numerical performance of the implementations.

5.1 Subroutine 1: Dense Unit-Lower Triangular Matrix-Vector Multiplication

Trial Statistics

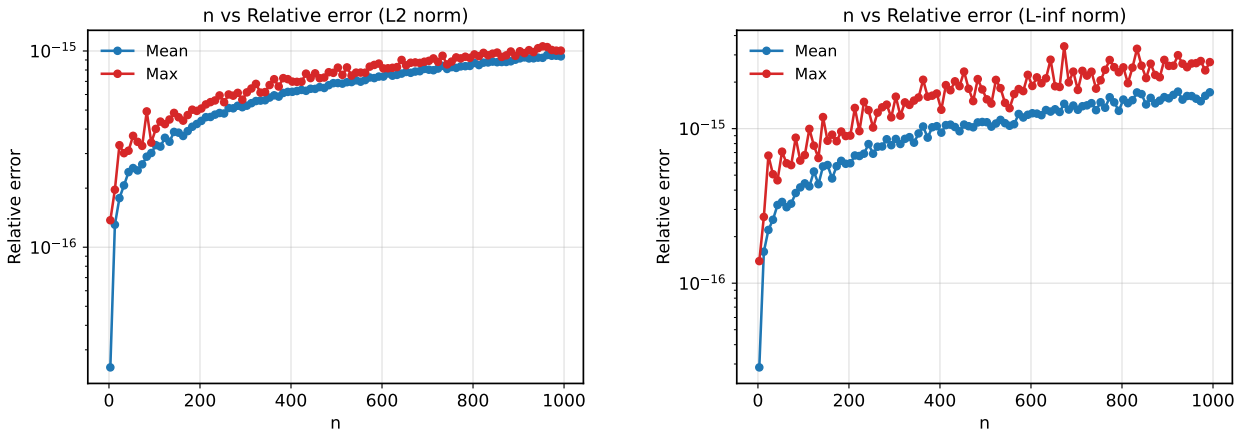
The subroutine was tested using 20 randomized trials for each matrix size, with the largest matrix dimension set to $n = 1000$. The computed results were compared with the dense matrix-vector product to evaluate accuracy. Relative errors were measured using the L_2 and L_∞ norms. The mean and maximum errors, overall mean error, and error variance were calculated to summarize the numerical performance. The resulting error statistics are summarized in Table 1, which shows the accuracy of the subroutine in both the L_2 and L_∞ norms.

Summary Statistics	L_2	L_∞
Mean of mean relative errors	6.400×10^{-16}	1.049×10^{-15}
Maximum of maximum relative errors	1.056×10^{-15}	3.407×10^{-15}
Overall mean relative error	6.400×10^{-16}	1.049×10^{-15}
Overall relative error variance	5.061×10^{-32}	2.724×10^{-31}

Table 1: Summary of relative errors for the unit lower triangular matrix-vector product.

Trend Plots of Mean and Max Relative Error

Figures 1a and 1b illustrate the trends in mean and maximum relative errors as the matrix dimension n increases, using the L_2 and L_∞ norms, respectively.



(a) L_2 norm

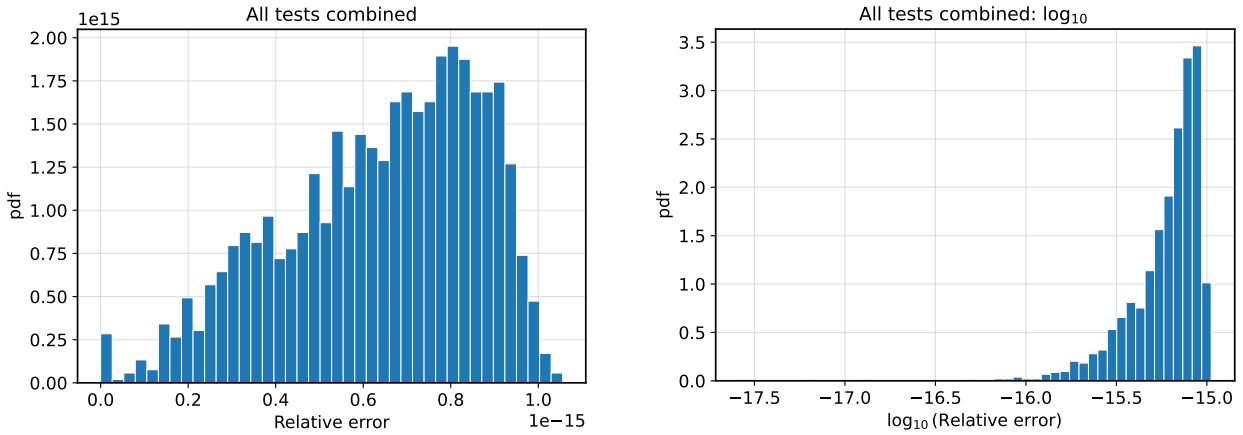
(b) L_∞ norm

Figure 1: Mean and maximum relative error trends for different matrix dimensions.

The figures show the mean relative error (blue curve) and maximum relative error (red curve) as functions of the matrix dimension n for both the L_2 and L_∞ norms. The errors remain on the order of 10^{-15} throughout the tested range, indicating that the computed results closely match the dense reference computations. Since floating-point rounding errors accumulate during the matrix-vector multiplication, a slight increase in error with matrix size is expected. However, the observed errors remain within a few multiples of machine precision ($u \approx 10^{-16}$). These results demonstrate that the subroutine maintains high numerical accuracy for matrix dimensions up to $n = 1000$.

Distribution of Relative Errors

Figure 2a presents the histogram of relative errors across all randomized trials, showing that the errors are concentrated within a narrow range. Figure 2b shows the same distribution on a logarithmic scale, making the magnitude of the errors easier to examine. The observed errors are generally on the order of 10^{-15}, indicating that the computed results closely agree with the dense reference computations. These results demonstrate that the subroutine maintains high numerical accuracy across the tested matrix sizes.



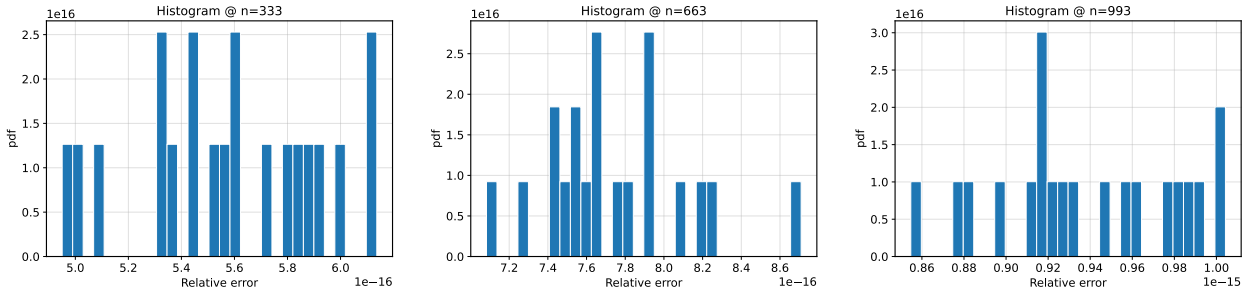
(a) Relative error distribution.

(b) Logarithmic relative error distribution.

Figure 2: Distribution of relative errors across all randomized trials.

Error Distributions for Per-n Matrix Sizes

Figures 3a–3c show the relative error distributions for three selected matrix sizes. These histograms allow us to examine how the distribution of numerical errors changes as the matrix dimension increases.



(a) $n = 333$

(b) $n = 663$

(c) $n = 993$

Figure 3: Relative error distributions for selected matrix sizes.

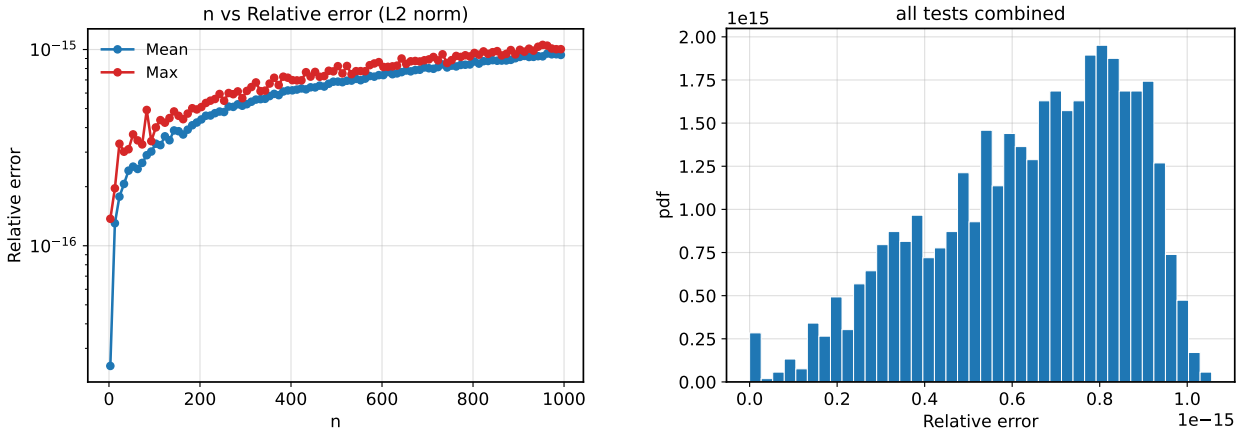
5.2 Subroutine 2: Dense Unit-Lower Triangular Matrix-Vector Multiplication with CSR storage

Subroutine 2 performs the same matrix–vector multiplication as Subroutine 1, using a unit lower-triangular matrix stored in compressed sparse row (CSR) format. Since both subroutines carry out the same mathematical operation, their results are nearly identical, with relative errors remaining at machine precision across all tested matrix sizes. Both methods are also affected by similar floating-point rounding errors. The primary advantage of CSR storage is its lower memory requirement, which is $O(n)$ for the strictly lower-triangular entries,

compared with $O(n^2)$ for dense storage. Therefore, only a representative selection of figures is presented for Subroutine 2 to demonstrate its numerical consistency with Subroutine 1 and highlight the memory efficiency of CSR storage.

Relative Error Trends and Distribution

Figure 4a- 4b shows the relative error behavior of Subroutine 2 for $n = 1000$. The mean and maximum relative errors exhibit behavior similar to that of Subroutine 1, with errors remaining close to machine precision. The histogram further confirms that the relative errors are tightly concentrated around 10^{-16}, indicating consistent numerical accuracy across all tests.



(a) Relative error in the L_2 norm

(b) Distribution of relative errors across all tests

Figure 4: Relative error analysis for Subroutine 2 with $n = 1000$.

5.3 Subroutine 3: Two-Subdiagonal Unit-Lower Matrix–Vector Multiplication

Subroutine 3 computes $w = Lv$, where L is a unit lower-triangular banded matrix with only two nonzero subdiagonals. Unlike Subroutines 1 and 2, which involve row-wise accumulations whose lengths increase with the row index, Subroutine 3 requires at most two off-diagonal terms per row, in addition to the diagonal contribution. Consequently, the rounding-error bound is governed by a fixed factor $\gamma_2 \approx 2u$, making it essentially independent of the matrix dimension n. This explains why the error plots for Subroutine 3 exhibit little variation as n increases, whereas Subroutines 1 and 2 show a mild increase in error due to their longer row-wise summations.

Trial Statistics

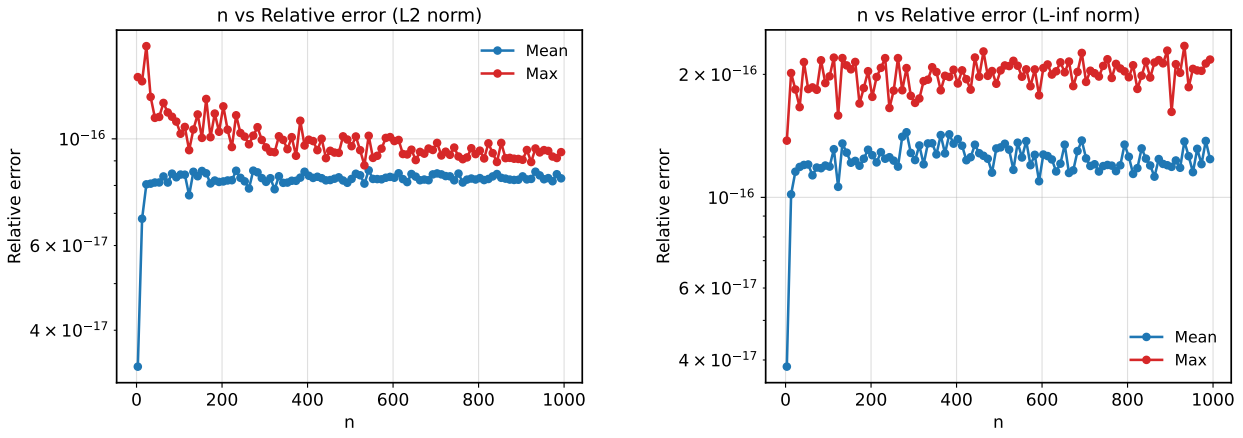
The subroutine was also evaluated using randomized trials for matrix dimensions up to $n = 1000$. Its numerical accuracy was assessed by measuring relative errors in the L_2 and L_∞ norms. The mean of the mean relative errors, maximum of the maximum relative errors, overall mean relative error, and overall relative error variance were computed to characterize its numerical performance. The resulting statistics are presented in Table 2, showing that the relative errors remain on the order of 10^{-16}, indicating high numerical accuracy in both norms.

Summary Statistics	L_2	L_∞
Mean of mean relative errors	8.214×10^{-17}	1.240×10^{-16}
Maximum of maximum relative errors	1.559×10^{-16}	2.347×10^{-16}
Overall mean relative error	8.214×10^{-17}	1.240×10^{-16}
Overall relative error variance	1.406×10^{-34}	1.510×10^{-33}

Table 2: Summary of relative errors for the unit lower-triangular matrix-vector product.

Trend Plots of Mean and Max Relative Error

Figures 5a and 5b show the mean and maximum relative error versus n up to $n = 1000$. In both norms the errors stay at machine precision, between about 4×10^{-17} and 2.3×10^{-16}, and show no upward trend with n. After a short initial rise at very small n, the 2-norm mean stays near 8×10^{-17} and the maximum decreases toward it. In the ∞-norm, the mean stays near 1.2×10^{-16} and the maximum fluctuates around 2×10^{-16}. This is expected, since each entry of w needs at most two products and two additions, so its rounding error is bounded independently of n. In Subroutines 1 and 2, by contrast, the row sums grow with i and the error creeps upward with n.

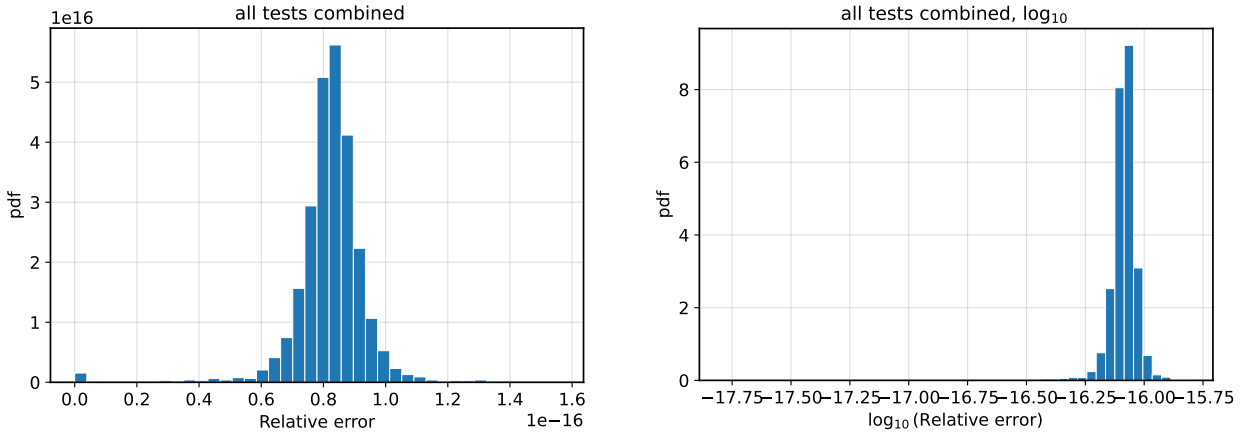


(a) L_2 norm (b) L_∞ norm

Figure 5: Mean and maximum relative error trends for different matrix dimensions

Distribution of Relative Errors

Figures 6a and 6b show the distribution of the 2-norm relative error pooled over all trials and sizes up to $n = 1000$. The errors are tightly concentrated: most values lie between about 0.6×10^{-16} and 1.0×10^{-16}, with a single sharp peak near 0.8×10^{-16}, below the machine epsilon of 2.2×10^{-16}. The right tail is very thin and ends near 1.3×10^{-16}, so no trial shows a large error. The small bar at zero comes from trials with essentially no rounding error, which occur at very small n where few operations are involved. These zero values are excluded from the $\log_{10}$ histogram. There the distribution is a narrow peak centered near $\log_{10}(\text{rel err}) \approx -16.1$, spanning less than half an order of magnitude, with only a faint left tail toward -16.5. This narrow, symmetric shape agrees with the flat trend plots: because each entry of w needs at most two products and two additions, the rounding error is bounded independently of n, and the spread comes only from the random draws of L and v.



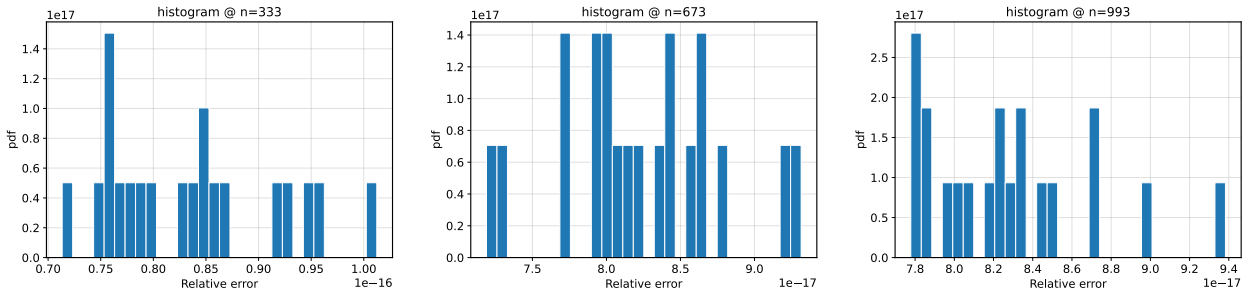
(a) Relative error distribution.

(b) Logarithmic relative error distribution.

Figure 6: Distribution of relative errors across all randomized trials.

Error Distributions for Per-n Matrix Sizes

Figure 7 shows the 2-norm relative error at $n = 333$, 673, and 993. At every size the 20 trial errors lie in a narrow band around 8×10^{-17}, below machine epsilon. The spread is widest at $n = 333$ (about 0.7 to 1.0×10^{-16}) and narrows at $n = 673$ and $n = 993$ (about 7.2 to 9.4×10^{-17}), while the center does not move. The bars look uneven because each histogram contains only 20 samples. The nearly identical location and spread at all three sizes agree with the flat trend plots and show that the error does not depend on n.



(a) $n = 333$ (b) $n = 673$ (c) $n = 993$

Figure 7: Relative error distributions for selected matrix sizes.

5.4 Subroutine 4: Packed LU Product

This subroutine computes the matrix product $M = LU$, where L is unit lower-triangular and U is upper-triangular, with both matrices represented in a single packed $n \times n$ array A. The strictly lower-triangular entries of A store the subdiagonal elements of L, while the diagonal and upper-triangular entries represent U. The unit diagonal of L is implicit. In contrast to Subroutine 3, which exploits a banded structure to achieve $O(n)$ computational complexity, Subroutine 4 explicitly reconstructs $M = LU$ using nested loops over the indices i, j, and k, requiring $O(n^3)$ operations. Consequently, the computational cost increases rapidly with the matrix dimension. The runtime becomes substantial at $n = 500$, while the computation at $n = 1000$ remains theoretically feasible but practically inefficient. Therefore, the corresponding figures are presented only up to $n = 500$.

Trial Statistics

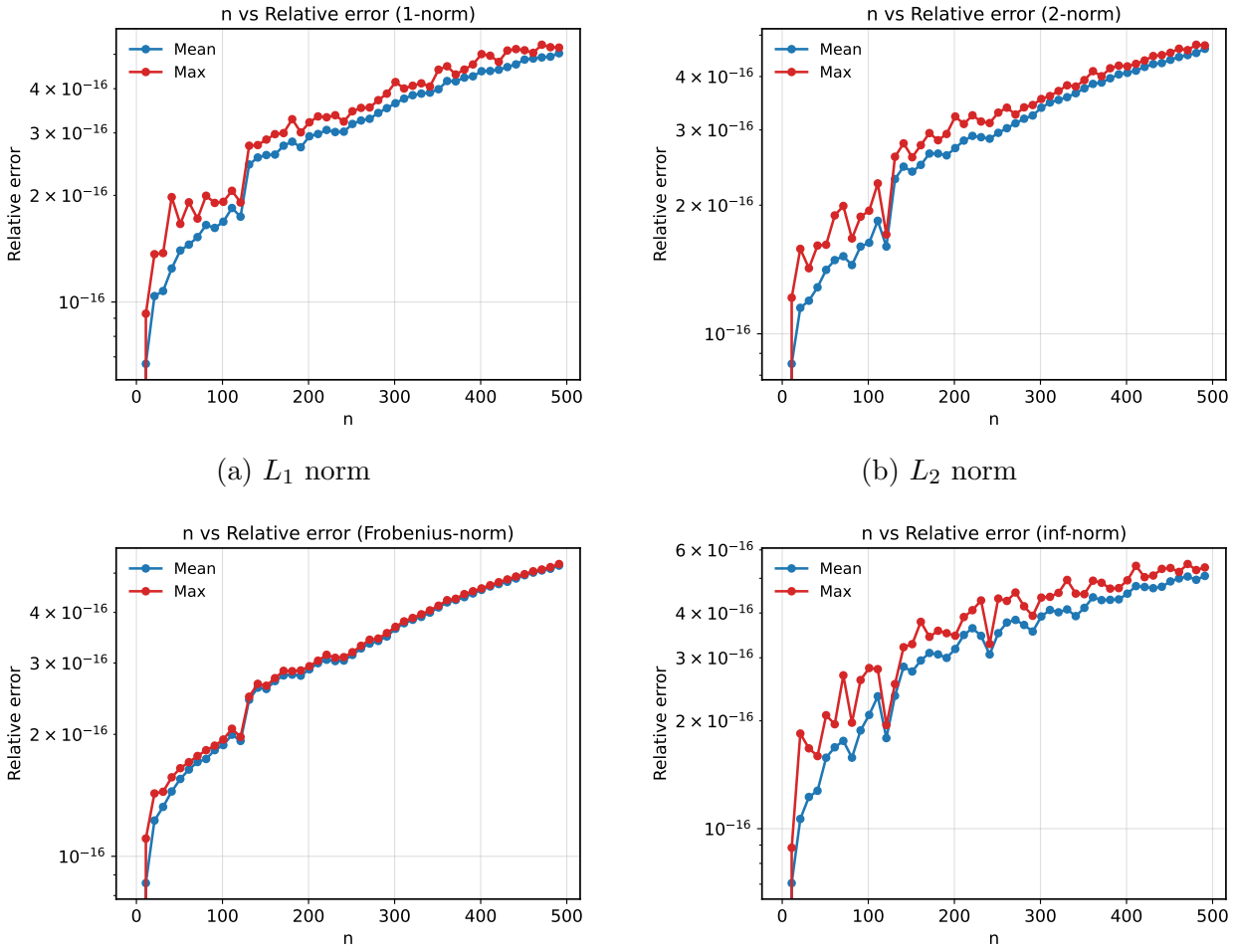
Table 3 summarizes the mean and maximum relative errors across all runs. For all four norms, the errors remain close to machine precision, confirming the numerical accuracy and stability of the packed LU product. The consistently small error values and variances indicate that the computation maintains reliable accuracy across the tested cases.

Summary Statistics	$\boldsymbol{L_1}$	$\boldsymbol{L_\infty}$	**Frobenius**	$\boldsymbol{L_2}$
Mean of mean rel. errors	3.090×10^{-16}	3.284×10^{-16}	3.180×10^{-16}	2.898×10^{-16}
Max of max rel. errors	5.322×10^{-16}	5.481×10^{-16}	5.267×10^{-16}	4.748×10^{-16}
All-tests rel. error mean	3.090×10^{-16}	3.284×10^{-16}	3.180×10^{-16}	2.898×10^{-16}
All-tests rel. error variance	1.680×10^{-32}	1.713×10^{-32}	1.639×10^{-32}	1.321×10^{-32}

Table 3: Summary statistics of relative errors for different norms.

Trend Plots of Mean and Max Relative Error

Figure 8 presents the mean and maximum relative errors of Subroutine 4 for the L_1, L_2, L_∞, and Frobenius norms as the matrix dimension increases up to $n = 500$. Unlike Subroutine 3, where the error remained essentially constant with n, the errors in Subroutine 4 exhibit mild variation as the matrix dimension increases, since each matrix entry may involve summations over more terms. Nevertheless, the error magnitudes remain close to machine precision, indicating high numerical accuracy. The Frobenius norm is particularly useful for analyzing matrix multiplication because it captures the overall error across all matrix entries rather than focusing on individual entries or the largest deviation. Thus, it provides a comprehensive measure of the global error in the computed product, complementing the other norms.



(a) L_1 norm

(b) L_2 norm

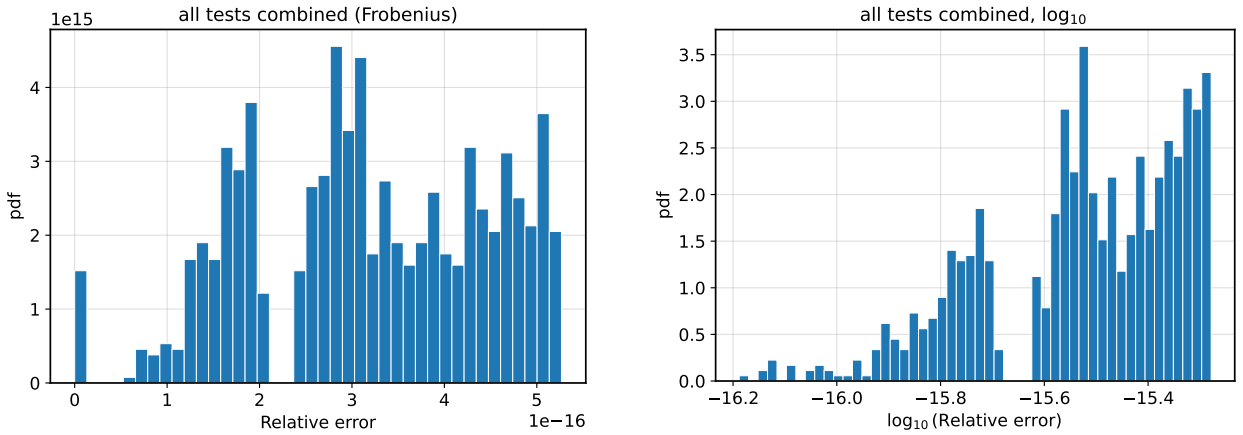
(c) Frobenius norm

(d) L_∞ norm

Figure 8: Mean and maximum relative errors of for different matrix dimensions and norms.

Distribution of Relative Errors

Figures 9a and 9b show the distribution of relative errors for all test cases using the Frobenius norm, presented on linear and logarithmic scales, respectively. The errors are concentrated in the order of 10^{-16}, indicating that the computed matrix product maintains high numerical accuracy. The logarithmic histogram provides a clearer view of small variations among error values that are difficult to distinguish on the linear scale. Overall, these results support the numerical stability of Subroutine 4 across the tested cases.



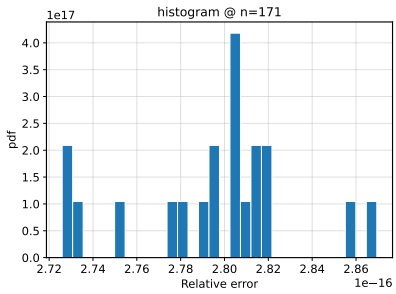
(a) Relative error distribution.

(b) Logarithmic relative error distribution.

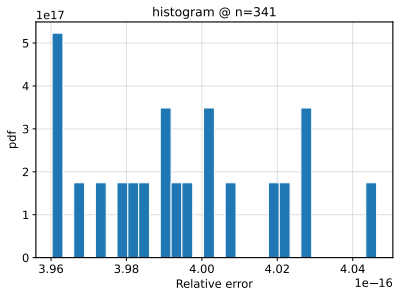
Figure 9: Distribution of relative errors across all randomized trials.

Error Distributions for Per-n Matrix Sizes

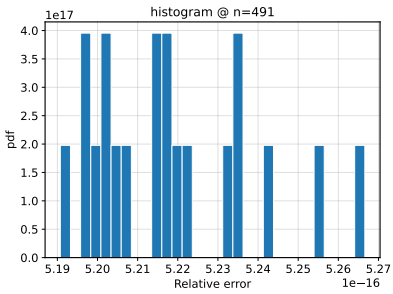
Figure 10 presents the distributions of relative errors for Subroutine 4 at matrix dimensions $n = 171$, $n = 341$, and $n = 491$. The histograms show that the errors are concentrated within narrow ranges, with magnitudes of approximately 2.8×10^{-16}, 4.0×10^{-16}, and 5.2×10^{-16}, respectively. This indicates a gradual increase in relative error as the matrix dimension grows, which is consistent with the accumulation of floating-point rounding errors during matrix multiplication. Nevertheless, the errors remain close to machine precision, demonstrating that Subroutine 4 maintains high numerical accuracy across the tested matrix sizes.



(a) $n = 171$



(b) $n = 341$



(c) $n = 491$

Figure 10: Relative error distributions for selected matrix sizes.

6 Comparison of the Four Subroutines

Overall, all four subroutines demonstrate numerical accuracy near machine precision, although their computational costs and error trends differ according to their matrix structures and algorithms. Subroutine 1 requires $O(n^2)$ operations and storage, with a mild increase in rounding error as n grows due to the increasing length of row-wise summations. Subroutine 2 performs essentially the same arithmetic and achieves comparable accuracy, while its storage requirements can be reduced for sparse matrices; however, its complexity remains $O(n^2)$ in the dense case. Subroutine 3 is the most computationally efficient, requiring $O(n)$ operations and storage. Since each row involves at most two off-diagonal terms, its rounding error remains nearly independent of n, resulting in a more consistent error distribution. In contrast, Subroutine 4 reconstructs $M = LU$ using explicit triple loops, leading to $O(n^3)$ computational complexity. Although its numerical errors also remain near machine precision, its rapidly increasing runtime limits its feasibility for large matrix dimensions, such as $n = 1000$.

7 Conclusions

All four subroutines maintained numerical stability, with relative errors remaining close to machine precision, although their computational efficiencies differed considerably. These results highlight the importance of adapting algorithms to the underlying storage formats and sparsity structures to improve efficiency without compromising numerical accuracy. Among the four approaches, Subroutine 3 offers the best combination of accuracy and scalability, whereas Subroutine 4 demonstrates the computational limitations of dense algorithms with cubic complexity.